EE1021 – Lectures 12 & 13 Study Sheet

Topics: MATLAB Basics, Matrix Manipulation, Control Flow (Loops/Conditions), Plotting & Visualization.
Goal: Prepare for the upcoming quiz by practicing these core skills.

Part A – Arrays & Matrices

1. Matrix Construction & Indexing

Create a script named `task1_matrices.m` to perform the following:

1. Create a matrix `A` (4 rows, 5 columns) where the elements are integers 1 to 20 filling the matrix row by row.
   - *Hint:* You can create it manually or manipulate a vector.
2. Extract the entire **3rd row** into a variable named `row_3`.
3. Extract the **last two columns** of the matrix into a variable named `cols_last`.
4. Change the value at row 2, column 4 to `0`.
5. Using a single command, squaring **every element** in `A` (element-wise) and store the result in `A_sq`.

2. Element-wise Operations vs Matrix Math

Create a script `task2_calculation.m`:

1. Define a vector `x` from -2π to 2π with 100 points.
2. Calculate `y1 = sin(x)`.
3. Calculate `y2 = x .* cos(x)`. *Note the dot!*
4. Calculate `y3` as the simplified polynomial $y = x^2 - 4x + 3$. Ensure you use element-wise powers (`.^`).

Part B – Control Flow

3. Loop Logic

Create a function or script `task3_loops.m`:

1. Create a vector `V` containing 20 random integers between 1 and 50 using `randi`.
2. Use a `for` loop to iterate through `V`.
3. Inside the loop, check if the current number is **even**.
   - If it is even, print "Number X is Even".
   - If it is odd, print "Number X is Odd".
4. (Optional challenge) Calculate the sum of all elements in `V` using a loop (do not use `sum()`).

4. While Loop Threshold

1. Start with a variable `balance = 100`.
2. Use a `while` loop that runs as long as `balance` is strictly positive (`> 0`).
3. In each iteration, subtract a random integer between 5 and 15 from `balance`.
4. Display the new balance at each step.

Part C – Visualization

5. Multi-line Plotting

Create a script `task5_plot.m`:

1. Define `t` as a time vector from 0 to 5 seconds (step 0.01).
2. Calculate two signals:
   - $s_1 = e^{-t} \cdot \sin(2\pi t)$
   - $s_2 = e^{-0.5t} \cdot \cos(2\pi t)$
3. Plot both on the **same figure** (use `hold on`).
4. Style the plot:
   - s_1: Blue solid line (`'b-'`), LineWidth 2.
   - s_2: Red dashed line (`'r--'`), LineWidth 2.
5. Add a **title** "Damped Oscillations", **x-label** "Time (s)", and **y-label** "Amplitude".
6. Add a **legend** identifying the signals.

6. Subplots & Chart Types

Create a script `task6_gallery.m` that generates a single figure with a **2x2 layout** of subplots:

- **Top-Left (1):** A **bar chart** of 5 arbitrary values (e.g., student grades). Title it "Grades".
- **Top-Right (2):** A **scatter plot** of 50 random x and y points. Use filled red circles. Title it "Random Scatter".
- **Bottom-Left (3):** A **histogram** of 1000 random numbers (use `randn`). Title it "Normal Distribution".
- **Bottom-Right (4):** An **image** or **contour plot**.
  - Create a grid (e.g., `Z = magic(10)` or a calculated function).
  - Display it using `imagesc` or `contourf`.
  - Title it "Heatmap".

Part D – Data Types

7. Strings & Cell Arrays

Create a script `task7_datatypes.m`:

1. Create a string variable `s1 = "Hello"` and `s2 = "MATLAB"`. (Note: Double quotes create string arrays in modern MATLAB, single quotes create char arrays. Use what you learned in class, likely single quotes for simple text or double for "string" type).
2. Combine them into a message `msg` saying "Hello MATLAB" (use concatenation or `plus`).
3. Create a **Cell Array** named `C` containing three elements:
   - The integer `42`.
   - The string/char `'EE1021'`.
   - A vector `[1 2 3]`.
4. Extract the **third element** (the vector) from `C` and store it in `v_out`.
5. Extract the string `'EE1021'` from `C`.

8. Structures

Create a script `task8_structs.m`:

1. Create a **Structure** named `student` with the following fields:
   - `name`: "Ali"
   - `id`: 12345
   - `scores`: A vector `[85 90 88]`
2. Add a new field `average` to the existing structure, calculating it as the mean of the `scores` field.
3. Display the student's name and calculated average.

Tip: Remember that MATLAB is 1-indexed. The first element of a vector `v` is `v(1)`, not `v(0)`.